Patrón SAGA con Temporal

Contenido

1. Introducción 3
1.1 Escenarios que se prueban 3
2. Requisitos 4
2.1 Software 4
2.2 Puertos utilizados 4
3. Instalación 5
3.1 Java y Maven 5
3.2 Temporal CLI (servidor de desarrollo) 5
3.3 Obtener el proyecto 5
3.4 Permitir la ejecución de scripts de PowerShell 5
4. Descripción de la orquestación 6
4.1 Arquitectura 6
4.2 Flujo de la saga 6
4.3 Decisiones de diseño 6
4.4 Resultado y estado consultable 7
5. Descripción de los proyectos 8
5.1 orquestador 8
5.2 Microservicios 9
5.3 scripts 10
6. Cómo compilar los proyectos 11
6.1 Compilar todo (sin pruebas) 11
6.2 Compilar y ejecutar las pruebas automatizadas 11
6.3 Compilar un solo módulo 11
6.4 Compilar con el script 11
7. Cómo ejecutar las pruebas 12
7.1 Pruebas de punta a punta (con el servidor de desarrollo) 12
7.2 Pruebas automatizadas (JUnit) 12
8. Dónde ver el historial de las pruebas 14
8.1 Interfaz web de Temporal (recomendado) 14
8.2 Temporal CLI 14
8.3 API del orquestador 14
8.4 Logs de los procesos 14
8.5 Bases de datos H2 14
9. Pruebas: cómo se provocan y resultado esperado 15
9.0 Prueba 0: camino feliz (referencia) 15

9.1 Prueba 1: hotel sin cupo 15
9.2 Prueba 2: pago rechazado (no reintentable) 16
9.3 Prueba 3: fallo temporal del correo 17
9.4 Prueba 4: caída del worker a mitad de la saga 18
10. Solución de problemas 20

1. Introducción

Una agencia de viajes vende paquetes que combinan vuelo, hotel, alquiler de auto, pago y notificación al cliente. Cada uno de estos servicios es un microservicio independiente, desarrollado en Spring Boot y con su propia base de datos H2. No existe una transacción distribuida que abarque las cinco bases de datos, por lo que la consistencia se logra con el patrón SAGA: una secuencia de transacciones locales en la que cada paso tiene una acción compensatoria que deshace su efecto si la saga no puede completarse.

La saga se implementa como orquestación usando Temporal (SDK de Java) y su servidor de desarrollo local. Temporal guarda de forma durable el historial de cada ejecución, reintenta automáticamente los pasos que fallan de forma temporal y permite que la saga sobreviva a la caída del proceso que la ejecuta.

Requisito de negocio central: si algo falla, el cliente no puede quedar con reservas huérfanas ni con cobros sin servicio.

1.1 Escenarios que se prueban

Prueba	Escenario
0	Camino feliz (referencia): todos los servicios responden correctamente.
1	Hotel sin cupo: error de negocio en el segundo paso.
2	Pago rechazado: error de negocio no reintentable en el paso pivote.
3	Fallo temporal del correo: el servidor de correo responde 503 varias veces.
4	Caída del worker a mitad de la saga: se mata el proceso orquestador y se reinicia.

2. Requisitos

2.1 Software

Componente	Versión	Uso
Java (JDK)	17 o superior	Compilar y ejecutar los microservicios y el orquestador (probado con Temurin 17.0.14).
Apache Maven	3.9 o superior	Compilación del proyecto multi-módulo (probado con 3.9.9).
Temporal CLI	1.x (1.9.1)	Incluye el servidor de desarrollo local (Server 1.32, UI 2.54).
Windows PowerShell	5.1 o superior	Scripts de arranque y de pruebas (carpeta `scripts`).

Dependencias principales que Maven descarga automáticamente:

- Spring Boot 3.5.16 (web, data-jpa, validation).
- Temporal Java SDK `io.temporal:temporal-sdk:1.40.0` y `temporal-testing` (pruebas).
- Base de datos H2 (embebida, en archivo).

2.2 Puertos utilizados

Proceso	Puerto	Descripción
Temporal Server (gRPC)	7233	Servidor de desarrollo de Temporal; los workers y clientes se conectan aquí.
Temporal Web UI	8233	Interfaz web para ver workflows, historial de eventos y reintentos.
orquestador	8080	API REST de reservas + worker de Temporal.
servicio-vuelos	8081	Reservas de vuelo.
servicio-hoteles	8082	Reservas de hotel con control de cupos.
servicio-autos	8083	Alquiler de autos.
servicio-pagos	8084	Cobros y reembolsos (pasarela simulada).
servicio-notificaciones	8085	Envío de correos (simulado).

3. Instalación

3.1 Java y Maven

1. Instale un JDK 17 (por ejemplo Eclipse Temurin) y configure la variable `JAVA_HOME`.
2. Instale Apache Maven 3.9+ y agregue su carpeta `bin` al `PATH`.
3. Verifique en una terminal:

```
java -version
mvn -v
```

3.2 Temporal CLI (servidor de desarrollo)

1. Descargue el CLI para Windows desde: `https://temporal.download/cli/archive/latest?platform=windows&arch=amd64`
2. Descomprima el archivo ZIP; contiene `temporal.exe`.
3. Copie `temporal.exe` a una carpeta (por ejemplo `C:\temporal`) y agréguela al `PATH` del sistema.
4. Abra una terminal nueva y verifique la instalación:

```
temporal --version
# temporal version 1.9.1 (Server 1.32.0, UI 2.54.1)
```

En Linux/macOS se puede instalar con `brew install temporal` o con `curl -sSf https://temporal.download/cli.sh | sh`.

3.3 Obtener el proyecto

Copie la carpeta `proyecto` a su equipo. Su contenido se describe en la sección 5. No requiere instalar bases de datos: cada servicio crea su archivo H2 en `<servicio>/data/` al arrancar.

3.4 Permitir la ejecución de scripts de PowerShell

Si PowerShell bloquea los scripts (`no está firmado digitalmente` o `la ejecución de scripts está deshabilitada`), habilítelos solo para la sesión actual:

```
Set-ExecutionPolicy -Scope Process Bypass
```

4. Descripción de la orquestación

4.1 Arquitectura

El cliente invoca la API del **orquestador**, que inicia un *workflow* de Temporal. El worker, que corre dentro del mismo proceso orquestador, ejecuta el workflow y llama a los microservicios mediante *actividades* (llamadas HTTP). Temporal registra cada evento (actividad iniciada, completada, fallida, timer, etc.) en su historial durable.

4.2 Flujo de la saga

El workflow `ReservaViajeWorkflowImpl` ejecuta los pasos en este orden. El identificador del workflow (`reserva-xxxxxxxx`) es el **sagaId** que viaja a todos los servicios.

#	Paso (acción)	Compensación	Política de reintentos
1	Reservar vuelo	Cancelar vuelo	5 intentos; backoff 1 s × 2, máx. 10 s
2	Reservar hotel (descuenta cupo)	Cancelar hotel (libera cupo)	5 intentos; backoff 1 s × 2, máx. 10 s
3	Reservar auto	Cancelar auto	5 intentos; backoff 1 s × 2, máx. 10 s
4	**Cobrar pago (pivote)**	Reembolsar pago	5 intentos; backoff 1 s × 2, máx. 10 s
5	Enviar correo de confirmación	No se compensa	10 intentos; backoff 2 s × 2, máx. 20 s

Las **compensaciones** se ejecutan en orden inverso (pago → auto → hotel → vuelo) y se reintentan **sin límite** (backoff hasta 30 s): una compensación nunca puede darse por vencida, porque abandonarla dejaría una reserva huérfana o un cobro sin servicio.

Cuando la saga se compensa, el workflow envía además un correo de **cancelación** al cliente (mejor esfuerzo).

4.3 Decisiones de diseño

Errores de negocio frente a fallos temporales

La clase `ClienteServicios` traduce la respuesta HTTP de cada microservicio a un fallo de Temporal:

- **4xx** (409 hotel sin cupo, 402 pago rechazado, 404…): `ApplicationFailure` **no reintentable**. Reintentar no cambiaría el resultado, así que la saga compensa de inmediato.
- **5xx** o servicio inaccesible (conexión rechazada, timeout): fallo **reintentable**; Temporal reintenta según la política de la tabla anterior.

La compensación se registra antes de invocar el paso

Si una petición llega al servicio pero la respuesta se pierde (timeout), el workflow no sabe si el paso se hizo. Por eso la compensación se agrega a la saga **antes** de llamar al servicio: en caso de fallo siempre se intenta deshacer, y como la cancelación es idempotente, no hay problema si el paso nunca ocurrió.

Idempotencia por sagaId

Todas las operaciones de los microservicios usan el `sagaId` como clave única. Repetir una reserva devuelve la misma reserva, repetir un cobro no cobra dos veces y repetir una cancelación no libera cupos dos veces. Esto hace seguros los reintentos automáticos de Temporal.

Lápidas (tombstones)

Si se pide cancelar algo que nunca se reservó, el servicio guarda un registro `CANCELADA` (en pagos, `ANULADO`). Si después llega tarde la petición de reserva original para esa saga, se rechaza con `RESERVA_CANCELADA`. Así se evita una reserva huérfana creada después de la compensación.

Transacción pivote

El **pago** es el punto de no retorno. Antes del pago, cualquier fallo compensa la saga. Después del pago, la reserva queda confirmada; el correo es un paso **reintentable** que no deshace la reserva. Si el correo fallara definitivamente, la saga termina `CONFIRMADA` con `notificacionEnviada = false`.

Durabilidad ante la caída del worker

El estado de la saga no vive en la memoria del worker sino en el historial de Temporal. Si el proceso muere, al levantarse un worker nuevo Temporal le entrega la ejecución; el worker **reproduce (replay)** el historial para reconstruir el estado sin volver a llamar a los servicios ya ejecutados, y continúa desde el siguiente paso. Las pausas entre pasos usan `Workflow.sleep`, un temporizador durable del servidor.

Concurrencia en cupos de hotel

La reserva de hotel toma un **bloqueo pesimista** (`PESSIMISTIC_WRITE`) sobre la fila del hotel, de modo que dos sagas simultáneas no puedan tomar el último cupo.

4.4 Resultado y estado consultable

El workflow devuelve un `ResultadoReserva` con: `sagaId`, `estado` (`CONFIRMADA` o `CANCELADA`), identificadores de vuelo, hotel, auto y pago, `notificacionEnviada` y `motivo`. Además expone una *query* `estado()` con el paso actual y el historial legible de la saga.

5. Descripción de los proyectos

El proyecto es un **multi-módulo Maven**. El `pom.xml` raíz hereda de Spring Boot y declara seis módulos:

```
proyecto/
├── pom.xml                       POM padre (Spring Boot 3.5.16, Temporal 1.40.0)
├── README.md
├── orquestador/                  Worker Temporal + API REST (:8080)
├── servicio-vuelos/              (:8081)  BD H2: data/vuelos-db
├── servicio-hoteles/             (:8082)  BD H2: data/hoteles-db
├── servicio-autos/               (:8083)  BD H2: data/autos-db
├── servicio-pagos/               (:8084)  BD H2: data/pagos-db
├── servicio-notificaciones/      (:8085)  BD H2: data/notificaciones-db
└── scripts/                      Scripts PowerShell de arranque y pruebas
```

5.1 orquestador

Paquete `co.edu.unicauca.saga.orquestador`.

Clase	Responsabilidad
workflow/ReservaViajeWorkflow	Interfaz del workflow: método `reservar` y query `estado`.
workflow/ReservaViajeWorkflowImpl	Lógica de la saga: pasos, compensaciones (`io.temporal.workflow.Saga`), políticas de reintento, pivote y notificación.
actividades/ServiciosViajeActivities(Impl)	Actividades de reservar/cancelar vuelo, hotel, auto y cobrar/reembolsar pago.
actividades/NotificacionActivities(Impl)	Actividad de envío de correo.
actividades/ClienteServicios	Cliente HTTP (`RestClient`) que traduce 4xx a fallo no reintentable y 5xx a reintentable.
config/TemporalConfig	Beans de conexión: `WorkflowServiceStubs`, `WorkflowClient`, `WorkerFactory`.
config/IniciadorWorker	Registra workflow y actividades en la cola `RESERVA_VIAJES_TQ` y arranca el worker.
api/ReservaController	API REST para iniciar la saga y consultar estado/resultado.
modelo/*	`SolicitudReserva`, `ResultadoReserva`, `EstadoSaga`.

Configuración (`application.yml`): `temporal.target` (localhost:7233), `temporal.task-queue`, `temporal.worker-habilitado` (si es `false` la instancia solo expone la API) y las URL de los servicios.

API REST del orquestador

Método	Ruta	Descripción
POST	/api/reservas	Inicia la saga; responde 202 con el `sagaId`.
GET	/api/reservas/{sagaId}	Paso actual e historial de la saga (query de Temporal).

Método	Ruta	Descripción
GET	/api/reservas/{sagaId}/resultado?segundos=30	Espera el resultado final; 202 si sigue en curso.

Ejemplo de cuerpo de `POST /api/reservas`:

```
{ "cliente": "Ana Torres", "email": "ana@correo.com",
  "origen": "PPN", "destino": "CTG",
  "fechaIda": "2026-12-01", "fechaRegreso": "2026-12-05",
  "codigoHotel": "HOTEL-CENTRO", "numeroTarjeta": "4111111111111111",
  "monto": 2500000, "pausaEntrePasosSegundos": 0 }
```

`pausaEntrePasosSegundos` solo se usa en pruebas: inserta un temporizador durable después de cada paso para poder tumbar el worker a mitad de la saga.

5.2 Microservicios

Los cinco comparten la misma estructura: entidad JPA, repositorio, servicio con la lógica (transaccional e idempotente), controlador REST y un manejador de errores que responde `{ "codigo", "mensaje" }` con el código HTTP correspondiente.

Servicio	Endpoints	Reglas de negocio / simulación
vuelos :8081	POST /api/vuelos/reservas POST /api/vuelos/reservas/{sagaId}/cancelar GET /api/vuelos/reservas	Reserva idempotente; cancelación con lápida.
hoteles :8082	POST /api/hoteles/reservas POST /api/hoteles/reservas/{sagaId}/cancelar GET /api/hoteles/reservas GET /api/hoteles	Catálogo inicial: HOTEL-CENTRO (10 cupos), HOTEL-PLAYA (5), **HOTEL-LLENO (0)**. Sin cupo → 409 `HOTEL_SIN_CUPO`. Cancelar libera el cupo.
autos :8083	POST /api/autos/reservas POST /api/autos/reservas/{sagaId}/cancelar GET /api/autos/reservas	Reserva idempotente; cancelación con lápida.
pagos :8084	POST /api/pagos POST /api/pagos/{sagaId}/reembolsar GET /api/pagos	Tarjeta **4000-0000-0000-0002** o monto > 20.000.000 → 402 `PAGO_RECHAZADO` (queda registrado). Estados: APROBADO, RECHAZADO, REEMBOLSADO, ANULADO.
notificaciones :8085	POST /api/notificaciones POST /api/notificaciones/simular-fallos?veces=N GET /api/notificaciones	Un correo por saga y tipo (CONFIRMACION / CANCELACION). `simular-fallos` hace que los próximos N envíos respondan 503.

Cada servicio tiene consola H2 en `http://localhost:808X/h2-console` (JDBC URL `jdbc:h2:file:./data/<servicio>-db`, usuario `sa`, sin contraseña).

5.3 scripts

Script	Función
comun.ps1	Funciones compartidas: iniciar saga, esperar resultado, mostrar historial y estado de las bases, verificaciones.
iniciar-todo.ps1 [-Compilar]	Compila (si se pide o faltan los JAR) y levanta los 5 servicios y el orquestador en segundo plano; logs en `logs/`.
detener-todo.ps1	Detiene servicios y orquestador (no detiene Temporal).
prueba-0-camino-feliz.ps1	Prueba de referencia.
prueba-1-hotel-sin-cupo.ps1	Escenario 1.
prueba-2-pago-rechazado.ps1	Escenario 2.
prueba-3-fallo-correo.ps1	Escenario 3.
prueba-4-caida-worker.ps1	Escenario 4.

6. Cómo compilar los proyectos

Todos los comandos se ejecutan desde la carpeta raíz `proyecto`.

6.1 Compilar todo (sin pruebas)

```
mvn clean package -DskipTests
```

Genera un JAR ejecutable por módulo, por ejemplo `servicio-hoteles/target/servicio-hoteles-1.0.0.jar` y `orquestador/target/orquestador-1.0.0.jar`. La primera vez descarga las dependencias (puede tardar varios minutos).

6.2 Compilar y ejecutar las pruebas automatizadas

```
mvn clean verify
```

Ejecuta `ReservaViajeWorkflowTest` (módulo orquestador) sobre un servidor Temporal **en memoria**; no requiere tener Temporal ni los servicios corriendo. Resultado esperado: `Tests run: 5, Failures: 0, Errors: 0`.

6.3 Compilar un solo módulo

```
mvn -pl servicio-pagos package -DskipTests
mvn -pl orquestador test
```

6.4 Compilar con el script

```
.\scripts\iniciar-todo.ps1 -Compilar
```

Compila todo y además arranca los servicios (ver sección 7).

7. Cómo ejecutar las pruebas

7.1 Pruebas de punta a punta (con el servidor de desarrollo)

Paso 1. En una terminal aparte, inicie el servidor de desarrollo de Temporal y déjelo abierto:

```
temporal server start-dev
```

El servidor queda en `localhost:7233` y la interfaz web en `http://localhost:8233`. La persistencia es en memoria: al cerrarlo se pierde el historial. Para conservarlo entre reinicios use `temporal server start-dev --db-filename temporal.db`.

Paso 2. En otra terminal PowerShell, desde la carpeta `proyecto`, levante los servicios:

```
Set-ExecutionPolicy -Scope Process Bypass
.\scripts\iniciar-todo.ps1 -Compilar
```

Debe terminar con `orquestador listo` y `Todo arriba`. Si avisa que no detecta Temporal en 7233, revise el paso 1.

Paso 3. Ejecute las pruebas (en cualquier orden):

```
.\scripts\prueba-0-camino-feliz.ps1
.\scripts\prueba-1-hotel-sin-cupo.ps1
.\scripts\prueba-2-pago-rechazado.ps1
.\scripts\prueba-3-fallo-correo.ps1
.\scripts\prueba-4-caida-worker.ps1
```

Cada script imprime: el `sagaId`, el historial de la saga, el estado que quedó en **cada base de datos** para ese sagaId y una lista de verificaciones `[OK]` / `[FALLO]`. El código de salida (`$LASTEXITCODE`) es el número de verificaciones fallidas (0 = éxito).

Paso 4. Al terminar:

```
.\scripts\detener-todo.ps1
# y Ctrl+C en la terminal de Temporal
```

> Para empezar desde cero (cupos y registros iniciales), detenga todo y borre las carpetas `data` de cada servicio y la carpeta `logs`.

7.2 Pruebas automatizadas (JUnit)

```
mvn -pl orquestador test
```

Prueba JUnit	Qué verifica
caminoFeliz	Saga CONFIRMADA, los cuatro pasos se ejecutan una vez y se envía la notificación.
hotelSinCupoCompensaElVuelo	Secuencia exacta: reservarVuelo, reservarHotel, cancelarHotel, cancelarVuelo; nunca cobra.
pagoRechazadoNoSeReintentaYCompensaTodo	Un solo intento de cobro y compensación en orden inverso.
falloTemporalDelCorreoSeReintentaSinCompensar	4 intentos de correo (3 fallos + 1 éxito) y ninguna compensación.
caidaDelWorkerSeRecuperaReproduciendoElHistorial	Reproduce el historial con `WorkflowReplayer` (lo que hace un worker

Prueba JUnit	Qué verifica
	nuevo tras una caída) sin repetir llamadas a servicios.

8. Dónde ver el historial de las pruebas

8.1 Interfaz web de Temporal

1. Abra `http://localhost:8233` en el navegador.
2. En el namespace **default**, la vista **Workflows** lista cada saga por su Workflow ID (`reserva-xxxxxxxx`), con su estado: Running, Completed, Failed.
3. Haga clic en una saga para ver su **Event History**: cada actividad programada, iniciada, completada o fallida, los temporizadores (`TimerStarted` / `TimerFired`) y el resultado final. Puede alternar entre las vistas *Timeline*, *Compact* y *JSON*.
4. Mientras una actividad se está reintentando, la sección **Pending Activities** muestra el número de intento (*Attempt*), el próximo reintento y el último error (por ejemplo `SMTP_NO_DISPONIBLE`).
5. En la pestaña **Queries** puede ejecutar la query `estado` para ver el paso actual y el historial legible.
6. Las entradas **Input** y **Result** muestran la solicitud y el `ResultadoReserva` en JSON.

> El servidor de desarrollo guarda el historial en memoria: se pierde al cerrarlo, salvo que se inicie con `--db-filename`.

8.2 Temporal CLI

```
temporal workflow list
temporal workflow show --workflow-id reserva-xxxxxxxx
temporal workflow describe --workflow-id reserva-xxxxxxxx
temporal workflow query --workflow-id reserva-xxxxxxxx --type estado
```

8.3 API del orquestador

```
Invoke-RestMethod http://localhost:8080/api/reservas/reserva-xxxxxxxx
Invoke-RestMethod http://localhost:8080/api/reservas/reserva-xxxxxxxx/resultado
```

La primera devuelve `paso` e `historial` (por ejemplo `VUELO_RESERVADO - VUELO-1`, `COMPENSANDO - HOTEL_SIN_CUPO: ...`, `CANCELADA`).

8.4 Logs de los procesos

Los scripts redirigen la salida de cada proceso a la carpeta `logs` del proyecto: `orquestador.log`, `servicio-vuelos.log`, etc. En `orquestador.log` cada llamada a un servicio aparece como `[sagaId] operacion (intento N)`, útil para contar reintentos:

```
Select-String -Path logs\orquestador.log -Pattern "reserva-xxxxxxxx"
```

Los logs de los servicios muestran las operaciones de negocio y compensaciones (por ejemplo `Compensación: reserva de hotel de saga ... CANCELADA, cupo liberado`).

8.5 Bases de datos H2

Para ver qué quedó persistido en cada servicio, use los endpoints `GET` de listado (sección 5.2) o la consola H2 de cada servicio (`http://localhost:808X/h2-console`). Ejemplo en la consola de pagos:

```
SELECT * FROM PAGO WHERE SAGA_ID = 'reserva-xxxxxxxx';
```

9. Pruebas: cómo se provocan y resultado esperado

9.0 Prueba 0: camino feliz (referencia)

Objetivo: comprobar que, sin fallos, la saga reserva todo, cobra una vez y notifica.

Cómo se provoca

- Solicitud normal: `codigoHotel = HOTEL-CENTRO`, tarjeta `4111111111111111`, monto 2.500.000.

```
.\scripts\prueba-0-camino-feliz.ps1
```

Resultado esperado

Historial de la saga:

```
VUELO_RESERVADO - VUELO-n
HOTEL_RESERVADO - HOTEL-n
AUTO_RESERVADO - AUTO-n
PAGO_APROBADO - PAGO-n
NOTIFICACION_ENVIADA - CONFIRMACION
CONFIRMADA
```

Estado en las bases de datos para ese sagaId:

Base de datos	Estado esperado
Vuelos	CONFIRMADA
Hoteles	CONFIRMADA (cupo de HOTEL-CENTRO disminuye en 1)
Autos	CONFIRMADA
Pagos	APROBADO
Notificaciones	CONFIRMACION

Verificaciones del script:

- Saga CONFIRMADA.
- Vuelo, hotel y auto confirmados; pago aprobado; correo de confirmación enviado.

En la interfaz de Temporal: workflow en estado **Completed**; cada actividad aparece una vez con `ActivityTaskCompleted`.

9.1 Prueba 1: hotel sin cupo

Objetivo: ante un error de negocio en el segundo paso, deshacer lo ya reservado (vuelo) sin reintentar y sin llegar a cobrar.

Cómo se provoca

- Se solicita el hotel `HOTEL-LLENO`, que la carga inicial crea con **0 cupos**.
- El servicio de hoteles responde **409** `HOTEL_SIN_CUPO`; el orquestador lo trata como **no reintentable**.

```
.\scripts\prueba-1-hotel-sin-cupo.ps1
```

Resultado esperado

Historial de la saga:

```
VUELO_RESERVADO - VUELO-n
```

```
COMPENSANDO - HOTEL_SIN_CUPO: Hoteles: El hotel Hotel Lleno Bogotá no tiene cupo
disponible
COMPENSADA - pasos previos revertidos en orden inverso
NOTIFICACION_ENVIADA - CANCELACION
CANCELADA - HOTEL_SIN_CUPO: ...
```

Estado en las bases de datos para ese sagaId:

Base de datos	Estado esperado
Vuelos	**CANCELADA** (compensado: sin reserva huérfana)
Hoteles	CANCELADA (lápida: la compensación se registra antes del paso)
Autos	Sin registros (nunca se llegó a reservar)
Pagos	Sin registros (**nunca se cobró**)
Notificaciones	CANCELACION

Verificaciones del script:

- Saga CANCELADA con motivo `HOTEL_SIN_CUPO`.
- Vuelo compensado.
- Nunca se reservó auto ni se cobró.

En la interfaz de Temporal: workflow **Completed** con resultado `estado = CANCELADA`. La actividad `ReservarHotel` falla **una sola vez** (attempt 1, non-retryable) y a continuación aparecen `CancelarHotel` y `CancelarVuelo`.

9.2 Prueba 2: pago rechazado (no reintentable)

Objetivo: ante el rechazo del pago, no reintentar el cobro y compensar todas las reservas en orden inverso.

Cómo se provoca

- Se usa la tarjeta de prueba `4000-0000-0000-0002`, que la pasarela simulada siempre rechaza por "Fondos insuficientes".
- El servicio de pagos responde **402** `PAGO_RECHAZADO` y guarda el intento como RECHAZADO; el orquestador lo trata como **no reintentable**.

```
.\scripts\prueba-2-pago-rechazado.ps1
```

Resultado esperado

Historial de la saga:

```
VUELO_RESERVADO - VUELO-n
HOTEL_RESERVADO - HOTEL-n
AUTO_RESERVADO - AUTO-n
COMPENSANDO - PAGO_RECHAZADO: Pagos: Fondos insuficientes
COMPENSADA - pasos previos revertidos en orden inverso
NOTIFICACION_ENVIADA - CANCELACION
CANCELADA - PAGO_RECHAZADO: Pagos: Fondos insuficientes
```

Estado en las bases de datos para ese sagaId:

Base de datos	Estado esperado
Vuelos	CANCELADA

Base de datos	Estado esperado
Hoteles	CANCELADA (cupo liberado)
Autos	CANCELADA
Pagos	**RECHAZADO (Fondos insuficientes)**: no hay cobro; el reembolso no cambia nada
Notificaciones	CANCELACION

Verificaciones del script:

- Saga CANCELADA con motivo `PAGO_RECHAZADO`.
- Cobro intentado **una sola vez** (se cuenta en `logs/orquestador.log`).
- Pago RECHAZADO; auto, hotel y vuelo compensados.

En la interfaz de Temporal: `CobrarPago` falla en el attempt 1 con `PAGO_RECHAZADO` (non-retryable); después se ejecutan en orden `ReembolsarPago`, `CancelarAuto`, `CancelarHotel` y `CancelarVuelo`.

9.3 Prueba 3: fallo temporal del correo

Objetivo: demostrar que un fallo transitorio se absorbe con reintentos y que, pasado el pivote (pago), la reserva no se deshace.

Cómo se provoca

- Antes de iniciar la saga, el script llama a `POST http://localhost:8085/api/notificaciones/simular-fallos?veces=3`.
- Los próximos 3 envíos de correo responden **503** `SMTP_NO_DISPONIBLE` (reintentable).
- Temporal reintenta con backoff exponencial (2 s, 4 s, 8 s); el 4.º intento tiene éxito (~14 s en total).

```
.\scripts\prueba-3-fallo-correo.ps1
```

Resultado esperado

Historial de la saga:

```
VUELO_RESERVADO - VUELO-n
HOTEL_RESERVADO - HOTEL-n
AUTO_RESERVADO - AUTO-n
PAGO_APROBADO - PAGO-n
NOTIFICACION_ENVIADA - CONFIRMACION
CONFIRMADA
```

Estado en las bases de datos para ese sagaId:

Base de datos	Estado esperado
Vuelos	CONFIRMADA
Hoteles	CONFIRMADA
Autos	CONFIRMADA
Pagos	APROBADO (**no se reembolsa**)
Notificaciones	CONFIRMACION (un solo registro)

Verificaciones del script:

- Saga CONFIRMADA: el fallo del correo no deshace la reserva.
- Correo enviado finalmente.
- **4 intentos** de envío: 3 fallidos + 1 exitoso.
- Pago sigue APROBADO.

En la interfaz de Temporal: durante la prueba, **Pending Activities** muestra `EnviarNotificacion` con *Attempt* 2, 3 y 4 y el último error `SMTP_NO_DISPONIBLE`. Al final, un único `ActivityTaskCompleted` para la notificación y ninguna actividad de cancelación.

9.4 Prueba 4: caída del worker a mitad de la saga

Objetivo: demostrar la durabilidad de la saga: si el proceso que la ejecuta muere, otro worker la retoma donde iba, sin repetir pasos ni cobrar dos veces.

Cómo se provoca

- La solicitud lleva `pausaEntrePasosSegundos = 8`: después de cada paso el workflow espera 8 s con un temporizador durable (`Workflow.sleep`).
- El script consulta el estado cada segundo y, cuando el paso es `HOTEL_RESERVADO`, **mata el proceso orquestador** (`Stop-Process -Force` sobre el PID del puerto 8080), lo que simula una caída abrupta.
- Espera 10 s con el worker caído (el auto aún no está reservado) y vuelve a iniciar el orquestador.
- Manualmente: inicie una saga con pausa, cierre la ventana o mate el proceso `java` del orquestador y vuelva a ejecutar `java -jar orquestador\target\orquestador-1.0.0.jar`.

```
.\scripts\prueba-4-caida-worker.ps1
```

Resultado esperado

Historial de la saga:

```
  paso actual: VUELO_RESERVADO   (varias veces)
  paso actual: HOTEL_RESERVADO
*** Worker ELIMINADO (PID nnnn) con la saga en HOTEL_RESERVADO ***
Mientras tanto en las bases: vuelo=CONFIRMADA, auto=
Reiniciando el worker...

VUELO_RESERVADO - VUELO-n
HOTEL_RESERVADO - HOTEL-n
AUTO_RESERVADO - AUTO-n
PAGO_APROBADO - PAGO-n
NOTIFICACION_ENVIADA - CONFIRMACION
CONFIRMADA
```

Estado en las bases de datos para ese sagaId:

Base de datos	Estado esperado
Vuelos	CONFIRMADA (**un único registro**)
Hoteles	CONFIRMADA
Autos	CONFIRMADA (reservado por el worker reiniciado)
Pagos	APROBADO (**un único cobro**)
Notificaciones	CONFIRMACION

Verificaciones del script:

- Saga CONFIRMADA tras reiniciar el worker.
- El log del worker nuevo **no** contiene `reservarVuelo` ni `reservarHotel`: se reconstruyeron por replay del historial.
- El worker nuevo continuó con `reservarAuto` y `cobrarPago`.
- Una sola reserva de vuelo y un único cobro APROBADO.

En la interfaz de Temporal: mientras el worker está caído, la saga sigue en estado **Running** (Temporal no la pierde). En el historial se ve `TimerStarted` tras el hotel; al reiniciar, un `WorkflowTaskScheduled/Started` atendido por una identidad de worker distinta (otro PID) y luego las actividades del auto y del pago. Las actividades de vuelo y hotel aparecen una sola vez.

10. Solución de problemas

Síntoma	Causa / solución
No se detecta Temporal en localhost:7233	El servidor de desarrollo no está corriendo: ejecute `temporal server start-dev` en otra terminal.
Nada escucha en el puerto 808X	El servicio no arrancó: revise `logs/<servicio>.log` y `.err.log` (puerto ocupado, JAR no compilado).
El script no se ejecuta por política de seguridad	`Set-ExecutionPolicy -Scope Process Bypass`.
`GET /api/reservas/{id}` falla tras reiniciar Temporal	El servidor de desarrollo guarda en memoria; la saga ya no existe. Use `--db-filename` para persistir.
Prueba 1 no da sin cupo / cupos alterados	HOTEL-LLENO siempre tiene 0 cupos. Para reiniciar cupos borre `servicio-hoteles/data`.
La query de estado no responde	Las queries requieren un worker activo: verifique que el orquestador esté arriba.